

GRAPH NOTES — Definitions to BFS & DFS

1. Definition of Graph

A **graph** is a non-linear data structure consisting of:

- **Vertices (Nodes):** Fundamental units of graph.
- **Edges:** Connections between vertices.

A graph is represented as:

$$G = (V, E)$$

where

- $V \rightarrow$ set of vertices
- $E \rightarrow$ set of edges

Example:

$$V = \{A, B, C\}$$

$$E = \{(A,B), (B,C)\}$$

2. Terminologies in Graph

✓ Vertex

A point/node in the graph.

✓ Edge

A link/connection between two vertices.

✓ Adjacent Vertex

If there is an edge between A and B, they are **adjacent**.

✓ Degree

Number of edges connected to a vertex.

- **Indegree** — No. of incoming edges
- **Outdegree** — No. of outgoing edges

✓ Path

A sequence of vertices connected by edges.

E.g., $A \rightarrow B \rightarrow C$

✓ Cycle

A path where the first and last vertices are the same.

✓ Connected Graph

There is at least one path between every pair of vertices.

✓ Weighted Graph

Edges have weights (cost, time, distance).

✓ Directed Graph (Digraph)

Edges have direction.

$A \rightarrow B$ is different from $B \rightarrow A$.

✓ Undirected Graph

Edges do not have direction.

✓ Sparse Graph

Very few edges compared to the number of vertices.

✓ Dense Graph

Large number of edges.

3. Matrix Representation of Graphs

Also called **Adjacency Matrix** representation.

It uses a **2D matrix** of size $n \times n$ (n = number of vertices).

✓ Rules

- If edge exists $\rightarrow$ store **1**
- If no edge $\rightarrow$ store **0**

Example Graph

A — B
B — C

Matrix:

	A	B	C
A	0	1	0
B	1	0	1
C	0	1	0

✓ Advantages

- Easy to implement.
- Good for dense graphs.
- Checking edge existence is **$O(1)$** .

✓ Disadvantages

- Uses $O(n^2)$ space $\rightarrow$ wastes memory for sparse graphs.
-

4. Adjacency List Representation

Graph is stored as a **list of lists**.

Each vertex stores a list of vertices connected to it.

Example:

A: B

B: A, C

C: B

✓ Advantages

- Efficient memory for sparse graphs.
- Uses $O(V + E)$ space.

✓ Disadvantages

- Checking if an edge exists may take $O(V)$ time.
-

5. Elementary Graph Operations

1 Insert Vertex

Add a new node to the graph.

2 Insert Edge

Add a connection between two nodes.

- For undirected graph → add for both nodes
- For directed graph → add only from source to destination

3 Delete Vertex

Remove the vertex and all edges connected to it.

4 Delete Edge

Remove the connection between two nodes.

5 Search

Find a node using traversal methods (BFS/DFS).

6 Display

Print the graph (matrix or list form).

6. Traversal Methods

Graph traversal means visiting each vertex in an order.

There are two primary graph traversal techniques:

A) Breadth-First Search (BFS)

✓ Definition

BFS visits nodes **level by level** from a starting node.
Uses **Queue (FIFO)**.

✓ Steps

1. Start from a source vertex.

2. Visit all neighbors.
3. Then move to the next level.

✓ Algorithm

```
BFS(start):  
    mark start as visited  
    enqueue(start)  
  
    while queue not empty:  
        node = dequeue()  
        visit(node)  
        for each adjacent of node:  
            if not visited:  
                mark visited  
                enqueue(adjacent)
```

✓ Applications

- Finding shortest path in unweighted graphs
- Web crawling
- Level-order traversal of trees
- Finding connected components

B) Depth-First Search (DFS)

✓ Definition

DFS explores a node **as far as possible along each branch**.
Uses **Stack (LIFO)** or recursion.

✓ Steps

1. Start from source.
2. Go deep until no unvisited neighbor.
3. Backtrack and continue.

✓ Algorithm

```
DFS(node):  
    mark node visited  
    for each adjacent of node:  
        if not visited:  
            DFS(adjacent)
```

✓ Applications

- Detecting cycles in graph
- Topological sorting
- Solving puzzles (maze)
- Finding connected components

✓ BFS vs DFS

Feature	BFS	DFS
Data Structure	Queue	Stack / Recursion
Traversal	Level-wise	Depth-wise
Shortest Path	Yes	No
Memory	More	Less
Good For	Shortest path	Deep search problems

